Korczak — מסמך המשכיות ופיתוח

Context Document for Development Continuity

עודכן: אפריל 2026

---

1. מה הפרויקט — תקציר לכל מי שמצטרף

Korczak הוא AI ומנווט — **ומכיר את המשתמש שלו לעומק** שמבין את מבנה הידע האקדמי לעומק אותו בתוך הידע כעמית שעובד איתך, לא ככלי חיפוש. הוא מבוסס על שני גרפים: **Knowledge Graph** שבונה מודל עמוק של האדם — לא רק מה הוא **User Graph**-שמתעדכן יומית ממאמרים אקדמיים, ו AI Navigator: יודע, אלא מי הוא, מה מניע אותו, מה הלחצים שלו, ואיך הוא חושב. מעל שני הגרפים Briefing שמלמד דרך שאלות, ומערכת Socratic Tutor, שמדבר מתוך הבנה של השטח **ושל האדם** תקופתית מותאמת אישית.

מחפש עבורך. קורצאק **מכיר אותך ומכיר את השטח** — ומביא לך AI **ההבדל במשפט אחד:** כל כלי בדיוק מה שאתה צריך ברגע הנכון, כולל דברים שלא ידעת לבקש.

מסמכי אפיון קיימים:

- `korczak-complete-vision.md` — חזון מלא, 12 חלקים, כל פיצ'ר עם רציונאל
- `korczak-2.0-prd.md` — PRD טכני עם קוד, system prompts, schemas, project structure

שנבחר: מבואות לאנתרופולוגיה (נבחר כי: תחום עם מחלוקות עמוקות, היסטוריה מורכבת, **MVP תחום** קשרים בינתחומיים — מבחן טוב ליכולות הגרף)

---

2. איפה אנחנו בתהליך

מה נעשה עד עכשיו

✅ הושלם — Prompt שלב 0: בדיקת

מטרה: לבדוק אם Claude API מסוגל לנתח מאמרים אקדמיים ברמה מספיקה לבניית Knowledge Graph.

מובנה prompt: **מה נבדק:** 3 עבודות יסוד באנתרופולוגיה נותחו עם

1. Malinowski — “Argonauts of the Western Pacific” (1922)
2. Geertz — “The Interpretation of Cultures” (1973)
3. Said — “Orientalism” (1978)

ה-Prompt שנבדק:

```
Analyze this academic work for a knowledge graph.

WORK: [title, author, year, abstract/description]

Extract in JSON:

1. CONCEPTS: Key concepts introduced or central to this work
   Format: [{name, type (theory/method/framework/phenomenon),
   definition, novelty_at_time (high/medium/low)}]

2. RELATIONSHIPS: How this work relates to other works/concepts
   Format: [{from, to, type (BUILDS_ON/CONTRADICTS/EXTENDS/
   APPLIES/ANALOGOUS_TO/RESPONDS_TO), confidence, explanation}]

3. CLAIMS: Central claims with evidence basis
   Format: [{claim, evidence_type, strength}]

4. HISTORICAL_SIGNIFICANCE: Role in the field's development
   Format: {paradigm_shift (true/false), influenced_fields[],
   controversy_generated, lasting_impact}
```

תוצאות:

- זיהוי מושגים: 9/10 — מושגים מרכזיים זוהו נכון בכל שלושת המקרים
- סוגי קשרים: 8/10 — ניואנס טוב (Malinowski את CONTRADICTS וגם EXTENDS גם Geertz)
- חיבורים בינתחומיים: 8/10 — Ryle→Geertz, Foucault→Said, Hermeneutics→Geertz — כולם נתפסו
- PREREQUISITE_FOR: 8/10 — סדר פדגוגי נכון (Said→Postcolonial, Geertz→Writing Culture)

בעיות שזוהו:

1. Confidence scores צריך — שרירותיים methodology מבוססת evidence count
2. ביקורת ומגבלות של עבודות — לא מספיק מיוצגות בניתוח

3. Entity resolution — לא נבדק עדיין (האם "Functionalism" מזוהה כאותו מושג בניתוחים שונים)
4. יודע" הרבה מאימון, על" LLM-מאמרים חדשים (2024-2025) — לא נבדקו. על מאמרים קנוניים ה חדשים — צריך בדיקה

מסקנה: עובד ברמת 8/10 על עבודות קנוניות. מספיק טוב להתחיל לבנות. צריך בדיקה נוספת Prompt על מאמרים חדשים.

מה צריך לעשות עכשיו — הצעדים הבאים

שלב 0.5: בדיקת מאמרים חדשים (טרם בוצע)

מטרה: לבדוק אם הניתוח עובד גם על מאמרים מ-2024-2025 שה-LLM לא "מכיר" מאימון.

איך:

1. שליפת 10 מאמרים חדשים באנתרופולוגיה מ-OpenAlex API
2. הרצת prompt הניתוח על כל אחד
3. בדיקה ידנית: האם מושגים, קשרים וטענות מדויקים?
4. השוואה לניתוח של עבודות קנוניות — האם יש ירידה באיכות?

Exit criteria: מאמרים מנותחים ברמה של 7/10 לפחות 7/10.

Script נדרש:

```
# test_new_papers.py
# 1. Fetch 10 recent anthropology papers from OpenAlex
# 2. Run analysis prompt on each via Claude API
# 3. Output results for manual review
# 4. Compare quality scores to canonical works
```

שלב 1: Graph + Navigator MVP

מטרה: גרף עובד של תחום אחד + Navigator שמנצח את ChatGPT.

משימות:

1. Supabase project setup (auth + DB schema)
2. OpenAlex ingestion pipeline (fetch → Claude analysis → DB)
3. Seed graph: 5,000 מאמרים באנתרופולוגיה

4. Entity resolution pipeline
5. Manual validation: 50 קשרים 100 ,צמתים
6. FastAPI backend + context builder
7. Navigator system prompt + WebSocket endpoint
8. Next.js chat interface (minimal)
9. 20-question benchmark: Korczak vs ChatGPT
10. Self-monitoring system (ראה חלק 4 במסמך זה)

Exit criteria: Navigator מנצח ChatGPT שאלות בתחום 15/20-ב.

---

3. החלטות ארכיטקטוניות שהתקבלו

3.1 DB: Supabase PostgreSQL בלבד (לא Neo4j)

החלטה: ב-MVP, הגרף חי ב-PostgreSQL עם pgvector. לא Neo4j.

רציונאל:

- Neo4j = עוד מורכבות, עוד עלות ($65+/חודש), עוד שרת
- PostgreSQL עם recursive CTEs מספיק ל-graph traversal 50-ב K צמתים
- pgvector כבר מובנה ב-Supabase — entity resolution + semantic search
- הכל במקום אחד: auth + user data + knowledge graph + embeddings

מתי עוברים ל-Neo4j: כשgraph traversal queries 100-ב כנראה — הופכים לצוואר בקבוק +K צמתים.

Schema מלא: ראה `korczak-2.0-prd.md` סעיף 9.

3.2 LLM: Claude API עם abstraction layer

החלטה: Claude Sonnet לbatch analysis, Claude Sonnet/Opus ל-Navigator/Tutor. עם abstraction layer שמאפשר החלפה.

רציונאל:

- Claude מצוין בניתוח אקדמי (הוכח בבדיקה)

- מגן מתלות בספק אחד Abstraction layer
- אגרסיבי מפחית קריאות Caching
- למשימות רוטיניות בעתיד (open source) מודלים קטנים

עלויות מוערכות:

- בניית גרף ראשוני (5K מאמרים): ~$200-300
- עדכון יומי: ~$90-300/חודש
- Navigator queries: ~$180/חודש (100 queries/day)

3.3 Frontend: Next.js 14 + D3.js

החלטה: Next.js לממשק, D3.js לויזואליזציה, shadcn/ui לרכיבים, Zustand ל-state.

רציונאל: Stack שמוכר (LocalTura), מהיר לפיתוח, Vercel deployment.

3.4 MVP-סילבוסים: לא ב

החלטה: לא לבנות על סילבוסים בשלב MVP.

רציונאל: אין API. רוב הסילבוסים לא ציבוריים. דורש גריפה ידנית או שותפויות. ה MVP-יבנה לסילבוסים. בלבד OpenAlex על (citation networks, concepts, funding, co-authorship).

סילבוסים = Phase 2 אחרי שיש מוצר עובד.

3.5 שפה: bilingual

החלטה: ה AI-עונה בשפה שהמשתמש כותב בה. מונחים מקצועיים באנגלית תמיד.

3.6 תחום MVP: אנתרופולוגיה

החלטה: מבואות לאנתרופולוגיה כתחום ראשון.

רציונאל: מחלוקות עמוקות (functionalism vs interpretivism vs postcolonialism), היסטוריה מורכבת, קשרים בינתחומיים (פילוסופיה, ספרות, היסטוריה, סוציולוגיה). מבחן אמיתי ליכולות הגרף.

הערה: לצורך מודל כלכלי, התחום הסופי למוצר מסחרי עשוי להיות אחר (AI Safety, Precision Medicine — שם יש יותר כסף). אנתרופולוגיה היא proof of concept-ל.

3.7 User Graph: "המשתמש כגוף ידע — "סוכן שמכיר אותך

הוא לא רק מפת ידע אקדמי של המשתמש — הוא **מודל מלא של האדם.** User Graph-**החלטה:** ה קורצאק לא רק יודע מה המשתמש יודע, אלא מי הוא, מה מניע אותו, מה הלחצים שלו, איך הוא חושב, ומה הוא צריך ברגע הזה.

רציונאל: ההבדל בין "כלי שמנווט בידע" ל-"עמית שמכיר אותך ומנווט אותך" הוא ההבדל בין $25/חודש שאנשים לא מוכנים לוותר עליו. אדם לא ישלם על חיפוש משופר. הוא כן ישלם על מישהו value-ל שמביא לו בדיוק מה שהוא צריך ברגע הנכון — כולל דברים שלא ידע לבקש.

שלוש שכבות ב-User Graph:

שכבה 1: Knowledge State (מה המשתמש יודע — MVP)

- רמת הבנה לכל מושג
- שזוהו misconceptions
- אקדמיים blind spots
- אנלוגיות שהמשתמש השתמש בהן

שכבה 2: Personal Context (מי האדם — V1)

- role: חוקר / דוקטורנט / סטודנט / R&D / סקרן
- מוסד, מחלקה, מנחה
- שלב: חוקר / כותב הצעה / אוסף נתונים / כותב
- נושא מחקר ספציפי
- קרובים deadlines
- תחומי עניין מחוץ לתחום העיקרי

שכבה 3: Behavioral Patterns (איך האדם פועל — V2)

- thinking_style: אנלוגי / כמותי / ויזואלי / תיאורטי / מעשי
- motivation: קריירה / בעיה ספציפית / deadline / סקרנות
- risk_tolerance: שמרן / חוקר (מחפש חיבורים בלתי צפויים)
- time_availability: 5 דקות ביום / שעה / כל היום
- emotional_patterns: מתוסכל מהר / סבלני / מתלהב ואז עוזב
- relationship_memory: נושאים שנדונו, שאלות פתוחות, הבטחות שקורצאק נתן

איך מתמלא — בלי שאלון:

הכל implicit מאינטראקציות:

- "אני דוקטורנט שנה שנייה" → role, year
- "יש לי הגשה בעוד חודשיים" → deadline, pressure
- "אין לי זמן, תגיד שורה תחתונה" → time_availability: low, preference: direct
- [15 שאלות על נושא צדדי] → curiosity_driven, risk_tolerance: explorer
- חיבור ORCID / Google Scholar → import (אופציונלי) אוטומטי

מה הסוכן עושה עם ההיכרות:

Proactive: לא מחכה לשאלות — יוזם. "ראיתי מאמר חדש שנוגע ישירות בשאלה שדיברנו עליה." "שלך בעוד 3 שבועות — חסרים לך מקורות בפרק המתודולוגיה ה-deadline."

Personalized depth: חוקרת בשבתון → חיבורים, actionable. דוקטורנט לחוץ → תשובות קצרות "הנה מה שהאקדמיה עושה שרלוונטי למה שאתם מפתחים." R&D מפתיעים, בלי לחץ. איש

Long-term memory: "מאז פורסמו 3 מאמרים שמשנים את התמונה. X לפני חודשיים שאלת על"

Advisor-aware: "המנחה שלך דוחפת לכיוון X. הנה 3 ארגומנטים חזקים בעד Y שתוכל להציג לה."

כלל קריטי: "Never be creepy"

המשתמש מרגיש שהתשובות מדויקות יותר, בלי שמרגיש "נצפה." — **invisible** ההיכרות צריכה להיות לעולם לא: "אני יודע שאתה..." אלא פשוט תשובות שמותאמות בצורה כל כך טבעית שזה מרגיש כמו שיחה עם מישהו שמכיר אותך.

שלבי הטמעה:

- **MVP:** שכבה 1 בלבד (knowledge state). Schema מוכן מראש 2+3 שכבות של.
- **V1 (אחרי user testing):** שכבה 2 (personal context) — role, topic, deadlines
- **V2:** שכבה 3 (patterns) — thinking style, motivation, emotional patterns
- **V3:** proactive suggestions, long-term memory, advisor awareness

Schema מורחב:

```
CREATE TABLE user_profiles (
  user_id UUID PRIMARY KEY REFERENCES auth.users(id),

  -- Identity
  display_name TEXT,
  language TEXT DEFAULT 'en',
```

```
  -- Context (Layer 2)
  role TEXT, -- 'undergrad','doctoral','postdoc','faculty','r_and_d','independent
  institution TEXT,
  department TEXT,
  advisor_name TEXT,
  research_topic TEXT,
  research_stage TEXT, -- 'exploring','proposal','data_collection','writing'
  upcoming_deadlines JSONB DEFAULT '[]',
  -- [{description, date, priority}]

  -- Connected accounts (optional)
  orcid_id TEXT,
  google_scholar_id TEXT,

  -- Patterns (Layer 3 — auto-updated)
  thinking_style JSONB DEFAULT '{
    "analogical": 0.5, "quantitative": 0.5,
    "visual": 0.5, "theoretical": 0.5, "practical": 0.5
  }',
  work_pattern JSONB DEFAULT '{
    "session_frequency": "unknown",
    "avg_session_length_min": 0,
    "question_style": "unknown",
    "depth_preference": "unknown"
  }',
  motivation TEXT, -- 'curiosity','deadline','career','problem_solving'
  risk_tolerance TEXT, -- 'conservative','moderate','explorer'
  time_availability TEXT, -- 'minimal','moderate','abundant'

  -- Learned preferences
  preferred_socratic_level INT DEFAULT 1,
  prefers_direct_answers BOOLEAN DEFAULT false,
  prefers_proactive_suggestions BOOLEAN DEFAULT true,

  -- Relationship memory
  topics_discussed JSONB DEFAULT '[]',
  -- [{topic, date, key_takeaway, open_questions}]
  promises_made JSONB DEFAULT '[]',
  -- [{what_korczak_promised, date, status: 'pending'|'fulfilled'|'expired'}]

  -- Meta
  session_count INT DEFAULT 0,
  first_seen TIMESTAMPTZ DEFAULT now(),
  last_seen TIMESTAMPTZ DEFAULT now()
);
```

Navigator System Prompt — עם User Context:

```
NAVIGATOR_SYSTEM_PROMPT = """
You are Korczak — a knowledge navigator who deeply understands
both the field AND the person you're talking to.

ABOUT THE USER:
{user_context}

You know this person. Not superficially — you understand their
work, their pressures, their interests, their patterns. Use this
to make every response specifically useful to THEM.

RULES:
1. Tailor everything to their specific situation.
   Don't say "researchers might find..."
   Say "given that you're working on [X], this means..."

2. Remember previous conversations naturally.
   "Last time we discussed [Y] — there's an update."

3. Be proactive when you see something relevant.
   "I know you didn't ask, but this connects to your
   [research topic/deadline/question from last time]."

4. Match their pace.
   Deadline mode → brief and actionable.
   Exploring mode → take your time, suggest connections.

5. If you know their advisor's perspective differs,
   help them prepare arguments for both sides.

6. Track promises. If you said "I'll keep an eye on [topic]"
   → follow up in the next briefing.

7. NEVER be creepy. Use what you know to be helpful,
   not to demonstrate how much you know about them.
   The knowledge should be invisible —
   the helpfulness should be obvious.

8. NEVER say "I know that you..." or "Based on your profile..."
   Just be helpful in a way that feels natural.

{graph_context}
"""
```

פרטיות — כללים מחייבים:

- **Encryption at rest** לכל ה-User Graph
- **User ownership מלא:** המשתמש יכול לראות, לערוך ולמחוק כל מידע
- **אף פעם** לא לשתף מידע של משתמש אחד עם אחר
- **שקיפות:** עמוד "מה קורצאק יודע עליי" שמציג הכל
- **Right to forget:** מחיקה מלאה בלחיצה
- Proactive suggestions = opt-in (ברירת מחדל: on, אבל ניתן לכיבוי)

---

4. מערכת ניטור, בקרה ותיקון עצמי

4.1 עקרון מנחה

המערכת צריכה לזהות, לדווח ולתקן בעיות בלי התערבות אדם כשאפשר, ולהתריע כשלא.

שלוש רמות:

1. **Auto-fix:** בעיה שהמערכת מזהה ומתקנת לבד
2. **Auto-alert:** בעיה שהמערכת מזהה אבל צריכה אישור אדם
3. **Auto-escalate:** בעיה שהמערכת לא מצליחה לטפל — עולה ל-admin

4.2 Graph Health Monitor

מה נבדק (רץ כל 6 שעות):

בדיקה 1: Consistency Checks

```
# graph_health/consistency.py

class ConsistencyChecker:
    """
    Runs every 6 hours. Checks graph for logical inconsistencies.
    """

    def check_circular_contradictions(self):
        """
        Find: A BUILDS_ON B, B BUILDS_ON C, C CONTRADICTS A
```

```
        This is a logical inconsistency that needs review.
        """
        query = """
        WITH RECURSIVE chain AS (
            SELECT source_id, target_id, rel_type, 1 as depth,
                   ARRAY[source_id] as path
            FROM relationships
            WHERE rel_type = 'BUILDS_ON'

            UNION ALL

            SELECT r.source_id, r.target_id, r.rel_type,
                   c.depth + 1, c.path || r.source_id
            FROM relationships r
            JOIN chain c ON r.source_id = c.target_id
            WHERE c.depth < 5
            AND r.source_id != ALL(c.path)
        )
        SELECT c.path, r.source_id, r.target_id
        FROM chain c
        JOIN relationships r
            ON r.source_id = c.target_id
            AND r.target_id = c.path[1]
            AND r.rel_type = 'CONTRADICTS'
        """
        # Action: AUTO-ALERT → add to review_queue

    def check_orphan_nodes(self):
        """
        Find nodes with zero incoming edges after 7+ days.
        Could be: legitimate new concept, or garbage.
        """
        query = """
        SELECT c.id, c.name, c.created_at
        FROM concepts c
        LEFT JOIN relationships r ON r.target_id = c.id
        WHERE r.id IS NULL
        AND c.created_at < now() - interval '7 days'
        """
        # Action: AUTO-ALERT if confidence < 0.6
        # Action: AUTO-FIX (mark as "unvalidated") if confidence < 0.4

    def check_duplicate_concepts(self):
        """
        Find concept pairs with very similar embeddings
        that might be duplicates.
        """
```

```
        query = """
        SELECT c1.id, c1.name, c2.id, c2.name,
               1 - (c1.embedding <=> c2.embedding) as similarity
        FROM concepts c1
        JOIN concepts c2 ON c1.id < c2.id
        WHERE 1 - (c1.embedding <=> c2.embedding) > 0.92
        """
        # similarity > 0.95: AUTO-FIX (merge, log)
        # similarity 0.92-0.95: AUTO-ALERT (merge candidate)

    def check_temporal_consistency(self):
        """
        A PRECEDED_BY B, but B.first_paper_year > A.first_paper_year
        """
        # Action: AUTO-FIX if clear, AUTO-ALERT if ambiguous

    def check_granularity(self):
        """
        Node with 10x more papers than siblings at same depth.
        Probably should be split.
        """
        query = """
        SELECT c.id, c.name, c.paper_count, c.depth, c.parent_id,
               avg_sibling.avg_count
        FROM concepts c
        JOIN (
            SELECT parent_id, AVG(paper_count) as avg_count
            FROM concepts
            GROUP BY parent_id
        ) avg_sibling ON c.parent_id = avg_sibling.parent_id
        WHERE c.paper_count > avg_sibling.avg_count * 10
        """
        # Action: AUTO-ALERT → "concept X may need splitting"
```

בדיקה 2: Vitals Freshness

```
# graph_health/vitals.py

class VitalsFreshnessChecker:
    """
    Ensures vitals (rate_of_change, trend, controversy_score)
    are up to date.
    """

    def check_stale_vitals(self):
        """
```

```
        Find nodes whose vitals haven't been recalculated
        in more than 48 hours.
        """
        query = """
        SELECT id, name, updated_at
        FROM concepts
        WHERE updated_at < now() - interval '48 hours'
        AND paper_count > 10
        """
        # Action: AUTO-FIX → trigger vitals recalculation

    def check_trend_accuracy(self):
        """
        Spot-check: does the calculated trend match reality?
        Sample 10 random nodes, verify trend against paper_count history.
        """
        # Run weekly
        # Action: If >3/10 incorrect → AUTO-ESCALATE
```

בדיקה 3: Confidence Decay

```
# graph_health/confidence.py

class ConfidenceDecayManager:
    """
    Relationships not reinforced by new evidence decay over time.
    """

    def apply_decay(self):
        """
        Run nightly. Relationships not supported by papers
        from last 2 years get confidence reduced by 0.02/month.
        Never goes below 0.3 (we don't delete, just note uncertainty).
        """
        query = """
        UPDATE relationships
        SET confidence = GREATEST(confidence - 0.02, 0.3),
            metadata = jsonb_set(
                metadata, '{decay_applied}', 'true'
            )
        WHERE updated_at < now() - interval '2 years'
        AND confidence > 0.3
        """
        # Action: AUTO-FIX (silent, logged)
        # If >100 relationships decayed in one night → AUTO-ALERT
```

4.3 Pipeline Health Monitor

מה נבדק (רץ עם כל pipeline run):

```
# pipeline_health/monitor.py

class PipelineHealthMonitor:
    """
    Monitors the daily ingestion pipeline for failures and anomalies.
    """

    def check_ingestion_success(self, report: PipelineReport):
        """
        After each daily pipeline run, check:
        """
        checks = {
            "papers_fetched": {
                "min": 10,  # at least 10 new papers/day expected
                "max": 5000,  # more than this = probably a bug
                "action_below": "AUTO-ALERT",
                "action_above": "AUTO-ALERT"
            },
            "analysis_success_rate": {
                "min": 0.90,  # 90% of papers should analyze successfully
                "action_below": "AUTO-ESCALATE"
            },
            "nodes_created": {
                "max": 500,  # shouldn't create 500 new concepts in a day
                "action_above": "AUTO-ALERT (possible entity resolution failure)"
            },
            "consistency_issues": {
                "max": 20,  # more than 20 issues/day = something wrong
                "action_above": "AUTO-ESCALATE"
            }
        }

        for metric, thresholds in checks.items():
            value = getattr(report, metric)
            if "min" in thresholds and value < thresholds["min"]:
                self.handle(thresholds["action_below"], metric, value)
            if "max" in thresholds and value > thresholds["max"]:
                self.handle(thresholds["action_above"], metric, value)

    def check_api_health(self):
        """
        Check that external APIs are responding.
```

```
        Run every hour.
        """
        apis = [
            {"name": "OpenAlex", "url": "https://api.openalex.org/works?sample=1"
             "expected_status": 200, "timeout": 10},
            {"name": "Claude API", "url": "https://api.anthropic.com/v1/messages"
             "method": "health_check", "timeout": 5},
            {"name": "Supabase", "url": "{SUPABASE_URL}/rest/v1/",
             "timeout": 5}
        ]
        for api in apis:
            try:
                response = httpx.get(api["url"], timeout=api["timeout"])
                if response.status_code != api["expected_status"]:
                    self.alert(f"{api['name']} returned {response.status_code}")
            except Exception as e:
                self.escalate(f"{api['name']} unreachable: {e}")

        # Action:
        # Single failure → AUTO-ALERT + retry in 5 min
        # 3 consecutive failures → AUTO-ESCALATE
        # If OpenAlex down → skip daily ingestion, AUTO-ALERT
        # If Claude API down → Navigator falls back to cached responses
```

4.4 Navigator/Tutor Quality Monitor

מה נבדק:

```
# quality/navigator_monitor.py

class NavigatorQualityMonitor:
    """
    Monitors quality of Navigator and Tutor responses.
    """

    def check_hallucination_signals(self, response: str,
                                    context: GraphContext):
        """
        After every Navigator response, check:
        Does the response contain claims not supported by the graph context?
        """
        check_prompt = f"""
        CONTEXT (from knowledge graph):
        {context.to_string()}
```

```
    NAVIGATOR RESPONSE:
    {response}

    Does the response contain any claims that are NOT
    supported by the context above?
    List them. If none, say "NONE".

    Return JSON: {{
        "unsupported_claims": [...],
        "severity": "none" | "minor" | "major"
    }}
    """
    # Run on 10% of responses (sampling)
    # If severity == "major":
    #   AUTO-FIX → flag response, don't cache
    #   AUTO-ALERT → log for prompt improvement
    # If >5% of sampled responses have "major": AUTO-ESCALATE

def check_source_accuracy(self, response: str):
    """
    If Navigator cites a paper — does that paper actually exist?
    """
    cited_dois = self.extract_dois(response)
    for doi in cited_dois:
        exists = self.check_paper_exists(doi)
        if not exists:
            # AUTO-FIX → remove citation from response
            # AUTO-ALERT → log hallucinated citation
            pass

def check_user_satisfaction(self):
    """
    Aggregate user feedback signals.
    Run daily.
    """
    query = """
    SELECT
        DATE(created_at) as day,
        COUNT(*) as total,
        COUNT(*) FILTER (WHERE user_feedback = 'negative') as negative,
        COUNT(*) FILTER (WHERE user_feedback = 'positive') as positive
    FROM interactions
    WHERE created_at > now() - interval '7 days'
    GROUP BY DATE(created_at)
    """
    # If negative_rate > 15% on any day → AUTO-ALERT
    # If negative_rate > 25% → AUTO-ESCALATE
```

```
        # If negative_rate trending up for 3 consecutive days → AUTO-ESCALATE

    def check_response_diversity(self):
        """
        Detect if Navigator is giving same/similar responses
        to different questions (sign of poor context building).
        """
        # Sample last 100 responses
        # Compute pairwise similarity
        # If any cluster of >5 very similar responses to different questions
        # → AUTO-ALERT (context builder may be broken)

    def run_benchmark_weekly(self):
        """
        Every week, automatically run the 20-question benchmark.
        Compare to baseline (ChatGPT answers stored from initial test).
        """
        # Run 20 standard questions through Navigator
        # Compare to stored ChatGPT answers
        # Score using LLM-as-judge
        # If win rate drops below 70% → AUTO-ESCALATE
        # Store results for trend tracking
```

4.5 User Graph Health

```
# quality/user_graph_monitor.py

class UserGraphMonitor:
    """
    Monitor User Graph quality and detect anomalies.
    """

    def check_understanding_inflation(self):
        """
        Detect if understanding_level is being inflated.
        If average understanding rises without corresponding
        engagement depth → scoring is too generous.
        """
        query = """
        SELECT
            AVG(understanding_level) as avg_understanding,
            AVG(engagement_count) as avg_engagement
        FROM user_knowledge
        WHERE last_engaged > now() - interval '7 days'
        """
```

```
        # If avg_understanding > 0.7 but avg_engagement < 3
        # → AUTO-ALERT (scoring may be inflated)

    def check_misconception_detection_rate(self):
        """
        If system never detects misconceptions → probably not detecting.
        """
        query = """
        SELECT COUNT(*) as total_interactions,
               COUNT(*) FILTER (
                   WHERE cardinality(misconceptions) > 0
               ) as with_misconceptions
        FROM user_knowledge
        """
        # If misconception_rate < 5% → AUTO-ALERT
        # (statistically improbable that no one has misconceptions)

    def check_cold_start_conversion(self):
        """
        Track: what % of zero-knowledge users reach light-profile?
        """
        query = """
        SELECT
            COUNT(*) FILTER (WHERE session_count >= 1) as visited,
            COUNT(*) FILTER (WHERE session_count >= 3) as light_profile,
            COUNT(*) FILTER (WHERE session_count >= 10) as rich_profile
        FROM user_profiles
        """
        # If light_profile / visited < 30% → AUTO-ALERT
        # (first experience may not be good enough)
```

4.6 Self-Healing Actions

```
# self_healing/actions.py

class SelfHealingEngine:
    """
    Automated responses to detected issues.
    """

    # === AUTO-FIX actions (no human needed) ===

    def merge_duplicate_concepts(self, concept_a_id, concept_b_id):
        """
        When two concepts have >0.95 embedding similarity:
```

```
    1. Keep the one with more relationships
    2. Transfer all relationships from the other
    3. Merge paper counts
    4. Log the merge in audit trail
    5. Mark merged-away concept as "merged_into: [winner]"
    """

def recalculate_stale_vitals(self, concept_ids: list):
    """
    When vitals are >48h old:
    1. Re-query OpenAlex for latest paper counts
    2. Recalculate rate_of_change
    3. Update trend classification
    4. Update controversy_score from recent CONTRADICTS edges
    """

def apply_confidence_decay(self):
    """Nightly: reduce confidence of unsupported relationships."""

def fallback_to_cache(self, service_name: str):
    """
    When an external API is down:
    1. Serve cached responses for Navigator queries
    2. Skip daily ingestion (don't create partial data)
    3. Show user notice: "Data may be up to [X] hours old"
    """

def fix_temporal_inconsistency(self, rel_id):
    """
    When A PRECEDED_BY B but B is newer:
    1. Check if it's a clear error (dates in metadata)
    2. If clear → swap direction
    3. If ambiguous → send to review queue
    """

def remove_hallucinated_citation(self, response_id, doi):
    """
    When a cited paper doesn't exist:
    1. Remove citation from cached response
    2. Log the hallucination
    3. Add to prompt improvement queue
    """

# === AUTO-ALERT actions (notify admin, no auto-fix) ===

def alert(self, issue_type: str, details: dict):
    """
```

```
        Send alert via:
        1. Internal dashboard (always)
        2. Email (if severity >= medium)
        3. Slack/Telegram (if severity >= high)

        Alert includes:
        - What was detected
        - When
        - Suggested action
        - Link to relevant data
        """
        alert = Alert(
            type=issue_type,
            severity=self.classify_severity(issue_type),
            details=details,
            suggested_action=self.suggest_action(issue_type),
            timestamp=datetime.utcnow()
        )
        self.store_alert(alert)

        if alert.severity >= Severity.MEDIUM:
            self.send_email(alert)
        if alert.severity >= Severity.HIGH:
            self.send_instant_message(alert)

    # === AUTO-ESCALATE actions (requires immediate attention) ===

    def escalate(self, issue_type: str, details: dict):
        """
        Critical issues:
        - Navigator quality dropping below threshold
        - Pipeline consistently failing
        - API costs spiking unexpectedly
        - User satisfaction plummeting

        Actions:
        1. All alert channels fired
        2. Feature flag: degrade gracefully
           (e.g., disable Tutor, keep Navigator)
        3. Log everything for post-mortem
        """
```

4.7 Health Dashboard

```
# Dashboard data structure — served via API endpoint
```

```
class HealthDashboard:
    """
    GET /api/admin/health
    Returns current system health status.
    """

    def get_status(self) -> dict:
        return {
            "timestamp": datetime.utcnow(),

            "graph_health": {
                "total_concepts": self.count_concepts(),
                "total_relationships": self.count_relationships(),
                "avg_confidence": self.avg_confidence(),
                "consistency_issues_open": self.open_issues(),
                "duplicates_pending": self.pending_merges(),
                "stale_vitals_count": self.stale_vitals(),
                "last_ingestion": self.last_pipeline_run(),
                "last_ingestion_status": "success" | "partial" | "failed",
                "papers_ingested_today": self.today_ingested(),
                "status": "healthy" | "degraded" | "critical"
            },

            "api_health": {
                "openalex": {"status": "up", "latency_ms": 120},
                "claude": {"status": "up", "latency_ms": 450},
                "supabase": {"status": "up", "latency_ms": 30}
            },

            "navigator_quality": {
                "hallucination_rate_7d": 0.02,
                "user_satisfaction_7d": 0.85,
                "benchmark_win_rate": 0.80,
                "avg_response_time_ms": 2100,
                "status": "healthy" | "degraded" | "critical"
            },

            "user_metrics": {
                "dau": 45,
                "wau": 120,
                "avg_session_length_min": 8.5,
                "cold_start_conversion": 0.35,
                "negative_feedback_rate": 0.08,
                "status": "healthy" | "watch" | "critical"
            },
```

```
        "costs": {
            "claude_api_today_usd": 8.50,
            "claude_api_mtd_usd": 180,
            "projected_monthly_usd": 350,
            "cost_per_user_monthly": 2.90,
            "status": "within_budget" | "warning" | "over_budget"
        },

        "alerts": {
            "open_critical": 0,
            "open_medium": 2,
            "open_low": 5,
            "resolved_today": 3
        }
    }
```

4.8 Scheduled Tasks Summary

Task	Frequency	Type	What it does
Daily Paper Ingestion	Every day 3AM UTC	Pipeline	Fetch new papers → analyze → update graph
Vitals Recalculation	Every 6 hours	Auto-fix	Recalculate trends, controversy, rates
Consistency Checks	Every 6 hours	Auto-alert	Find contradictions, orphans, duplicates
Confidence Decay	Nightly	Auto-fix	Reduce confidence of stale relationships
API Health Check	Every hour	Auto-alert/fix	Verify external APIs responding
Hallucination Sampling	Every 100 responses	Auto-alert	Check 10% of responses for unsupported claims
Source Verification	Every response	Auto-fix	Verify cited papers exist
User Satisfaction Check	Daily	Auto-alert	Aggregate feedback, detect trends
Weekly	Every Sunday	Auto-	Run 20 questions, compare to

Benchmark		escalate	baseline
Duplicate Detection	Daily	Auto-fix/alert	Find and merge similar concepts
Understanding Inflation	Weekly	Auto-alert	Check if user levels are inflated
Cost Monitoring	Hourly	Auto-alert	Track API spend, alert if over budget
Pipeline Report	After each run	Log	Store metrics for trend analysis

4.9 Graceful Degradation

When things go wrong, the system degrades gracefully instead of crashing:

Failure	Impact	Automatic Response
OpenAlex down	No new papers ingested	Skip ingestion, serve existing graph, alert
Claude API down	Navigator can't respond	Serve cached responses for common queries, show "limited mode"
Claude API slow	High latency	Queue non-urgent analyses, prioritize live user queries
Supabase down	No user data	Navigator works without personalization, alert
High API costs	Budget exceeded	Reduce sampling rate, cache more aggressively, alert
Graph quality drop	Bad user experience	Disable Tutor (higher quality bar), keep Navigator, alert
Hallucination spike	Trust erosion	Increase confidence threshold for claims, add disclaimers

---

5. בדיקת ה-20 שאלות — Benchmark

שאלות הבנצ'מרק לתחום האנתרופולוגיה

יש להריץ אותן מול ChatGPT/Claude (ללא גרף) לקבלת baseline, ואז מול Korczak Navigator.

שאלות מיקום (5):

1. "מה הזרמים המרכזיים באנתרופולוגיה תרבותית היום?"
2. "מה הויכוחים העיקריים באנתרופולוגיה של השנים האחרונות?"
3. "מה המצב של אנתרופולוגיה דיגיטלית כתת-תחום?"
4. "איך אנתרופולוגיה רפואית התפתחה מאז COVID?"
5. "מה הקשר בין אנתרופולוגיה לבינה מלאכותית?"

שאלות מחלוקת (3):

6. "על מה חלוקים באנתרופולוגיה לגבי הקונספט של 'תרבות'?"
7. "מה הויכוח סביב repatriation של אובייקטים מוזיאליים?"
8. "האם אנתרופולוגיה היא מדע או humanities?"

שאלות חיבור (3):

9. "מה הקשר בין אנתרופולוגיה כלכלית לכלכלה התנהגותית?"
10. "איך פוסט-קולוניאליזם באנתרופולוגיה מתחבר ל-critical race theory?"
11. "לפילוסופיה של המדע (STS) יש קשר בין אנתרופולוגיה של מדע?"

שאלות עדכניות (3):

12. "מי החוקרים העולים באנתרופולוגיה של טכנולוגיה?"
13. "מה המאמרים המשפיעים ביותר באנתרופולוגיה מ-2024?"
14. "מה הנושאים הכי 'חמים' באנתרופולוגיה עכשיו?"

שאלות מימון (2):

15. "מי מממן מחקר אנתרופולוגי? איך ההתפלגות השתנתה?"
16. "האם יש מגמה של מימון תעשייתי באנתרופולוגיה?"

שאלות blind spot (3 + 1 ולידציה):

17. "מה אנתרופולוגים מערביים מפספסים לגבי מחקר אנתרופולוגי לא-מערבי?"

18. "מה התחומים באנתרופולוגיה שהיו פעם חמים ומתו?"
19. "הנה רעיון: לחקור את האנתרופולוגיה של קהילות AI. נעשה?"
20. "אם הייתי מתחיל דוקטורט באנתרופולוגיה היום — מה הנישה הכי פתוחה?"

Scoring (1-10 ,לכל שאלה):

- **דיוק:** האם המידע נכון עובדתית?
- **עומק:** האם התשובה מעבר לשטחית?
- **עדכניות:** האם התשובה מבוססת על מידע עדכני?
- **תובנות:** האם יש משהו שלא ציפיתי לו?

חייב לנצח ב-15/20 שאלות Korczak (overall score).

---

6. הצעדים הבאים — סדר פעולה מדויק

```
☐ שלב 0.5: בדיקת מאמרים חדשים
  ├── Script: fetch 10 papers from OpenAlex (anthropology, 2024-2025)
  ├── Run analysis prompt on each
  ├── Manual review: are concepts/relationships accurate?
  └── Decision: proceed or improve prompt?

☐ שלב 1a: Infrastructure
  ├── Supabase project setup
  ├── DB schema (from korczak-2.0-prd.md section 9)
  ├── FastAPI skeleton
  ├── Next.js skeleton
  └── Docker compose for local dev

☐ שלב 1b: Graph Seeding
  ├── OpenAlex client (fetch papers by topic)
  ├── Claude batch analyzer (paper → JSON analysis)
  ├── Graph updater (JSON → DB inserts)
  ├── Entity resolver (embedding-based dedup)
  ├── Seed 5,000 anthropology papers
  └── Manual validation: 50 nodes, 100 edges

☐ שלב 1c: Navigator
  ├── Context builder (graph → LLM context)
  ├── Navigator system prompt
  ├── WebSocket chat endpoint
```

```
  ├── Basic chat UI
  ├── Run 20-question benchmark
  └── Decision: proceed or improve?

☐ 1 שלבd: Self-Monitoring v1
  ├── Consistency checker (circular, orphans, duplicates)
  ├── Pipeline health monitor
  ├── API health checks
  ├── Hallucination sampling
  ├── Health dashboard endpoint
  └── Alert system (email)

☐ 2 שלב: User Testing (only if step 1 passes)
  ├── 10 real users try Navigator
  ├── Collect feedback
  ├── Start capturing implicit user signals (role, topic from conversation)
  └── Iterate

☐ 3 שלב: Tutor + User Graph Layer 1 (only if step 2 is positive)
  ├── Mode detector
  ├── Level detector
  ├── Tutor system prompt
  ├── Prerequisite checker
  ├── User Graph: knowledge state tracking (what they know/don't know)
  └── Test with 5 users

☐ 3.5 שלב: User Graph Layer 2 — Personal Context
  ├── Implicit extraction: role, topic, institution from conversation
  ├── Optional: ORCID / Google Scholar connect
  ├── Deadline tracking
  ├── Context-aware Navigator (tailored to who they are, not just what they ask)
  ├── Conversation memory (topics discussed, open questions)
  └── Test: do users feel the difference? "this feels like it knows me"

☐ 4 שלב: Differentiation features
  ├── Controversy mapper
  ├── White space finder
  ├── Rising stars
  ├── Briefing engine (now personalized with User Graph)
  ├── Proactive suggestions ("you didn't ask, but...")
  └── Visualization v1

☐ 4.5 שלב: User Graph Layer 3 — Behavioral Patterns
  ├── Thinking style detection
  ├── Motivation classification
  ├── Pace/depth adaptation
  ├── Long-term memory and promise tracking
```

```
   ├── Advisor awareness
   └── "Never be creepy" testing with real users

☐ שלב 5: Beta launch
   ├── Landing page
   ├── Stripe
   ├── Privacy controls: "what Korczak knows about me" page
   ├── Full self-monitoring
   └── Launch
```

---

7. קבצים קיימים

קובץ	תוכן	שימוש
`korczak-complete-vision.md`	חזון מלא, כל פיצ'ר עם רציונאל	לכל מי שמצטרף
`korczak-2.0-prd.md`	PRD טכני: קוד, schemas, prompts, project structure	ל-Claude Code
`korczak-continuity.md`	**מסמך זה** — context, החלטות, איפה אנחנו, מה הבא	המשכיות בין sessions
`korczak-platform-*.md` (8 files)	v1 (Pre-pivot) אפיון מקורי	רקע היסטורי בלבד

---

8. שאלות פתוחות שטרם הוכרעו

1. **תחום MVP:** אנתרופולוגיה (PoC) vs AI Safety (commercial) — מתי עוברים?
2. **Hume integration timing:** כמה קריטי ל-MVP? אפשר לדחות ל-v2?
3. **Open source vs closed:** יתרון: community contributions. חיסרון: האם לפתוח את הגרף? moat נחלש
4. **Pricing validation:** willingness to pay עוד לא דיברנו עם אנשי R&D על
5. **Multi-language:** מתי להוסיף עברית כשפה נתמכת לגמרי (לא רק interface)?
6. **User Graph depth vs creepiness:** איפה הקו? כמה "היכרות" מרגישה מועילה וכמה מפחידה? צריך user research ספציפי על זה

7. **User Graph portability:** האם משתמש יכול לייצא את ה-User Graph שלו? להעביר ל-service אחר? זו שאלת אתיקה לא רק טכנולוגיה
8. **Proactive suggestions frequency:** כמה פעמים ביום/בשבוע קורצאק "מדבר" בלי שביקשו? צריך למצוא את ה-sweet spot
9. **Conversation memory retention:** כמה זמן לזכור שיחות ישנות? לנצח? חודש? המשתמש מחליט?
10. **Promise tracking liability:** אם קורצאק "מבטיח" לעקוב אחרי נושא ומפספס — מה ההשלכה? צריך expectation management

---

9. ציר הזמן של ההחלטות בשיחה הזו

תיעוד כרונולוגי של איך המוצר התפתח בשיחה:

שלב	מה עלה	מה השתנה
1	סקירת AI breakthroughs 2026	זיהוי World Models, Neuromorphic, Emotional AI כפרדיגמות חדשות
2	רעיונות למוצרים	"World Experience Studio", "Living Textbook", "Materials Playground"
3	ביקורת: "סוכנים לעסקים כבר יש"	ראה מה שאתה לא רואה — "Knowledge GPS" פיבוט לכיוון
4	ניתוח מסמכי קורצאק המקוריים	pivot הוא מוצר 2020, שוק צפוף, צריך v1 זיהוי: אפיון
5	pivot ל-Knowledge Graph + Navigator	הגרף הוא ה-backend, לא ה-frontend
6	ביקורת: ויזואליזציה 3D = gimmick	לא כממשק ראשי tool-ויזואליזציה כ zoom, מפה → 4 רמות
7	הרחבה: לא רק אקדמיה	אותה ארכיטקטורה לכל תחום — Domain Adapter pattern

8	User Graph כ-knowledge state	מיפוי מה המשתמש יודע/לא יודע
9	Socratic Tutor + Level Detection	למידה דרך שאלות, 4 רמות, anti-annoyance
10	בדיקת prompt על 3 מאמרים	8/10 — מספיק טוב ל-MVP
11	מערכת self-monitoring	5 שכבות validation, auto-fix/alert/escalate, graceful degradation
12	**User Graph = "סוכן שמכיר אותך"**	**שינוי מהותי: לא רק מה יודע אלא מי הוא. שלוש שכבות: knowledge, context, patterns. Proactive, personalized, advisor-aware. "Never be creepy."**